

Introduction to Sorting

For additional reading and resources, you can refer to <https://jeffe.cs.illinois.edu/teaching/algorithms/book/00-intro.pdf> and <https://jeffe.cs.illinois.edu/teaching/algorithms/book/01-recursion.pdf>

For this set of lecture notes in particular, I also referenced <https://courses.grainger.illinois.edu/cs277/fa2021/assets/lectures/slides/cs277-10-Sorting.pdf> which might be helpful for students.

Yesterday, we covered binary search where we find for a value in a list by cutting down the search space of our algorithm. However, for binary search to work, we learned that an array needs to be sorted. Today, we will start learning about some algorithms for sorting.

Why does sorting matter?

- Sorting makes searching easy!
- A set in sorted order is a canonical form!
- Sorting allows us to process items in a defined order!

Human sorting

Let's say we have the following random numbers:

```
[18, 4, 8, 0, 23, 6, 12, 2, 55, 7]
```

If we want to order these numbers from the SMALLEST number to the LARGEST number, how could we do that?

```
# Sorting the above numbers: [18, 4, 8, 0, 23, 6, 12, 2, 55, 7]
# [0, 2, 4, 7, 8, 12, 18, 23, 55]
```

One of the most straightforward ways to sort is to try and compare elements with one another and "bubble up" larger values in the array. This algorithm is also known as Bubble Sort.

Bubble Sort

For bubble sort, we will take the following approach:

1. Compare element i with element $i+1$
2. Swap them if element i is greater
3. Repeat until the array has been sorted

```
# bubble sort implementation
def bubblesort(arr):
    for j in range(len(arr)): #  $O(n) * O(n)$ 
        swapped = False
```

```

    for i in range(len(arr) - 1 - j):    #  $O(n)$ 
        if arr[i] > arr[i+1]:          #  $O(1)$ 
            arr[i], arr[i+1] = arr[i+1], arr[i]    #  $O(1)$ 
            swapped = True              #  $O(1)$ 
        if (swapped == False):
            break

arr_unsorted = [1,5,2,10,4,7,6]
bubblesort(arr_unsorted)
print(arr_unsorted)

[1, 2, 4, 5, 6, 7, 10]

```

What is the time complexity of bubble sort?

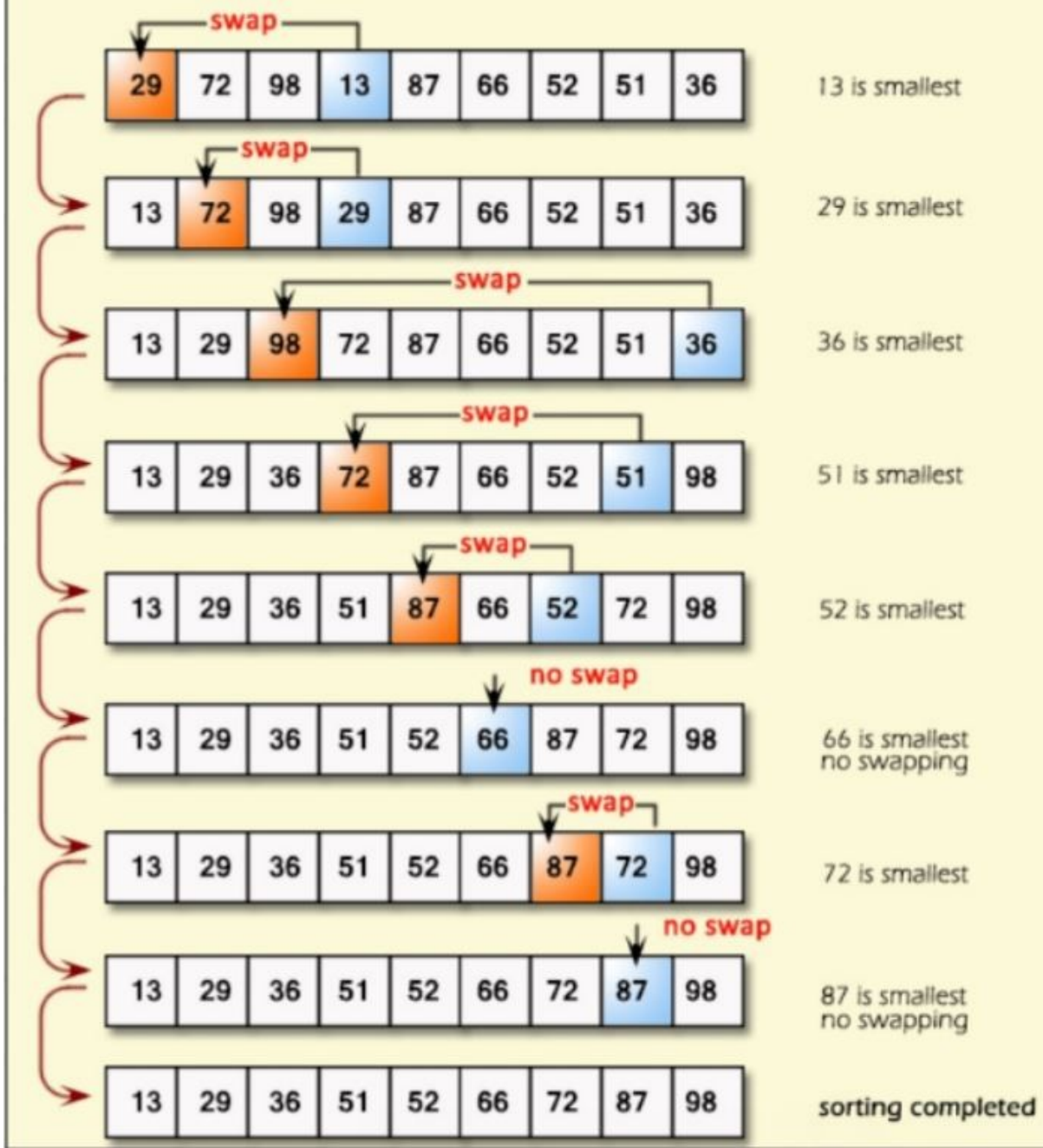
```
#  $O(N^2)$ 
```

Can bubble sort finish in much less than $O(N^2)$ time on some "good" `arr`, or does it always have to take $O(N^2)$ time?

Selection Sort

In selection sort, we can select the lowest value each time in the array and put it into a sorted section of the array. We want to keep track of where we insert the smallest element and the smallest element. Below is a visual representation:

Selection Sort



In person demonstration

Now, let's try to implement selection sort in code:

```
def selection_sort(arr): # another implementation of selection sort
    for i in range(len(arr)):
        for j in range(i + 1, len(arr)):
            if arr[j] < arr[i]:
                arr[i], arr[j] = arr[j], arr[i]

l_unsorted = [7, 2, 6, 0, 18, 4, 9]

selection_sort(l_unsorted)
print(l_unsorted)

[0, 2, 4, 6, 7, 9, 18]
```

The above code we wrote is an iterative implementation of selection sort. Similar to binary search, this code can also be written recursively! We won't cover this in class, but in your lab you will attempt to write a recursive selection sort.

Time Complexity of Selection Sort

Let's evaluate the function we wrote above for selection sort. We can evaluate this line by line:

```
# Selection sort function
def selection_sort(arr):
    for i in range(len(arr)): #  $O(n)$  *  $O(n)$ 
        smallest = i
        for j in range(i + 1, len(arr)): #  $O(n)$ 
            if arr[j] < arr[smallest]: #  $O(1)$ 
                smallest = j #  $O(1)$ 
        arr[i], arr[smallest] = arr[smallest], arr[i]
```

Can selection sort finish in much less than $O(N^2)$ time on some "good" `arr`, or does it always have to take $O(N^2)$ time?

Additional Notes

The bubble sort code and the selection sort code we wrote above are an IN PLACE implementation. This means we do not create a new array to store the sorted values, but instead do all operations inside the existing array.

The bubble sort code and selection sort code we wrote above are also a STABLE sort. This means for elements of the same value, they retain their order from the original input.

```
[2, 6, 8, 6, 9]
```